

AdaBoost Classifier

Boosting, Weak Learners, Stumps, Sample Weights, and Weighted Voting

CSE427: Machine Learning

Learning Objectives

By the end of this lecture, students should be able to:

- 1 Explain the idea of boosting.
- 2 Distinguish AdaBoost from Random Forest.
- 3 Explain weak learners and decision stumps.
- 4 Construct the first AdaBoost stump using Gini impurity.
- 5 Compute total error and amount of say.
- 6 Update sample weights after a weak learner.
- 7 Explain how AdaBoost combines weak learners using weighted voting.

Motivation: Why Do We Need Boosting?

Suppose we want to classify an email as **Spam** or **Not Spam**.

A few simple rules may be:

- If the email has promotional images, it may be spam.
- If the email contains many links, it may be spam.
- If the email says “You won prize money”, it may be spam.
- If the email comes from an official domain, it may not be spam.
- If the email comes from a known source, it may not be spam.

Question

Is any one of these rules individually strong enough?

Core Idea of Boosting

Boosting is an ensemble learning technique.

Main Idea

Boosting combines many weak learners to build one strong learner.

Weak Learners $\implies$ *Strong Learner*

- Each weak learner may perform only slightly better than random guessing.
- Boosting trains weak learners sequentially.
- Later learners focus more on the mistakes of earlier learners.

Types of Boosting Algorithms

Common boosting algorithms include:

- 1 AdaBoost
- 2 Gradient Boosting
- 3 XGBoost
- 4 LightGBM
- 5 CatBoost

This Lecture

We focus on **AdaBoost**, one of the earliest and most important boosting algorithms.

What is AdaBoost?

AdaBoost stands for **Adaptive Boosting**.

Definition

AdaBoost combines multiple weak classifiers into a single strong classifier.

- It was developed by Yoav Freund and Robert Schapire.
- It is mainly introduced for binary classification.
- It adapts by giving more attention to incorrectly classified samples.

$$\textit{AdaBoost} = \textit{Adaptive} + \textit{Boosting}$$

Weak Learner

A weak learner is a simple model that performs slightly better than random guessing.
For binary classification:

Random Guessing $\approx 50\%$

A weak learner may have accuracy:

55%, 60%, 65%

Important

A weak learner does not need to be very strong. AdaBoost improves performance by combining many weak learners.

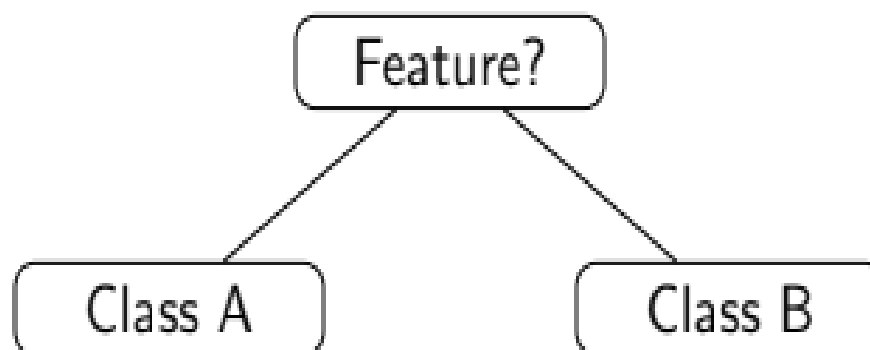
Decision Stump

In AdaBoost, the weak learner is often a **decision stump**.

Decision Stump

A decision stump is a decision tree with only one internal node and its leaf nodes.

Decision Stump = One-level Decision Tree



AdaBoost vs Random Forest

Property	Random Forest	AdaBoost
Training style	Parallel trees	Sequential stumps
Learners	Deep or medium trees	Usually decision stumps
Data emphasis	Bootstrap samples	Higher weight on wrong samples
Voting	Equal voting usually	Weighted voting
Order matters?	No	Yes
Main goal	Reduce variance	Reduce bias and focus on hard cases

Key Difference

Random Forest builds many independent trees. AdaBoost builds learners one after another, where each learner learns from the mistakes of previous learners.

AdaBoost Procedure

- 1 Assign equal weight to every training sample.
- 2 Train a weak learner using the current sample weights.
- 3 Find which samples are misclassified.
- 4 Increase the weights of misclassified samples.
- 5 Decrease the weights of correctly classified samples.
- 6 Train the next weak learner.
- 7 Combine all weak learners using weighted voting.

Why Sample Weights Matter

At the beginning, all samples are treated equally.

If there are N samples:

$$w_i = \frac{1}{N}$$

After each weak learner:

- wrongly classified samples get larger weights,
- correctly classified samples get smaller weights.

Meaning

A higher weight means the next weak learner should pay more attention to that sample.

AdaBoost Algorithm: Mathematical View

Given training data:

$$D = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$$

where:

$$y_i \in \{-1, +1\}$$

Initialize:

$$w_i = \frac{1}{N}, \quad i = 1, 2, \dots, N$$

For each weak learner $m = 1, 2, \dots, M$:

- ① Train weak classifier $G_m(x)$.
- ② Compute weighted error.
- ③ Compute amount of say α_m .
- ④ Update sample weights.

Weighted Error

The weighted error of weak learner G_m is:

$$err_m = \frac{\sum_{i=1}^N w_i I(y_i \neq G_m(x_i))}{\sum_{i=1}^N w_i}$$

- $I(y_i \neq G_m(x_i)) = 1$, if sample i is misclassified.
- $I(y_i \neq G_m(x_i)) = 0$, if sample i is correctly classified.

Simple Meaning

Total error is the sum of weights of incorrectly classified samples.

Amount of Say

Each weak learner receives an importance score called **amount of say**.

$$\alpha_m = \frac{1}{2} \ln \left(\frac{1 - \text{err}_m}{\text{err}_m} \right)$$

- If error is small, α_m is large.
- If error is close to 0.5, α_m is close to 0.
- If error is greater than 0.5, the learner is worse than random guessing.

Understanding Amount of Say

$$\alpha_m = \frac{1}{2} \ln \left(\frac{1 - \text{err}_m}{\text{err}_m} \right)$$

Error	Amount of Say	Meaning
Low	High positive	Good learner
0.5	0	Like random guessing
High	Negative	Worse than random

Important

A learner with lower error gets more influence in the final prediction.

Weight Update Rule

After computing α_m , update sample weights.

For a correctly classified sample:

$$w_i^{\text{new}} = w_i e^{-\alpha_m}$$

For an incorrectly classified sample:

$$w_i^{\text{new}} = w_i e^{\alpha_m}$$

Interpretation

Wrong samples receive larger weights, so the next stump focuses more on them.

Normalize the Weights

After updating weights, the weights may not sum to 1.
So, we normalize them:

$$w_i^{\text{normalized}} = \frac{w_i^{\text{new}}}{\sum_{j=1}^N w_j^{\text{new}}}$$

Why Normalize?

Normalization keeps the weights interpretable as proportions and makes weighted sampling easier.

Final AdaBoost Classifier

After training M weak learners, the final classifier is:

$$H(x) = \text{sign} \left(\sum_{m=1}^M \alpha_m G_m(x) \right)$$

- $G_m(x)$ is the prediction of weak learner m .
- α_m is the amount of say of weak learner m .

Key Point

AdaBoost does not use simple majority voting. It uses weighted voting.

Dataset: Play Tennis

We use the standard Play Tennis dataset.

Day	Outlook	Temperature	Humidity	Wind	Play Tennis
Day1	Sunny	Hot	High	Weak	No
Day2	Sunny	Hot	High	Strong	No
Day3	Overcast	Hot	High	Weak	Yes
Day4	Rain	Mild	High	Weak	Yes
Day5	Rain	Cool	Normal	Weak	Yes
Day6	Rain	Cool	Normal	Strong	No
Day7	Overcast	Cool	Normal	Strong	Yes
Day8	Sunny	Mild	High	Weak	No
Day9	Sunny	Cool	Normal	Weak	Yes
Day10	Rain	Mild	Normal	Weak	Yes
Day11	Sunny	Mild	Normal	Strong	Yes
Day12	Overcast	Mild	High	Strong	Yes
Day13	Overcast	Hot	Normal	Weak	Yes
Day14	Rain	Mild	High	Strong	No

Initial sample weight:

$$w_i = \frac{1}{14}$$

Step 1: Select the First Stump

To select the first stump, we calculate the weighted Gini impurity for each feature.
Candidate features:

Outlook, Temperature, Humidity, Wind

At the beginning, all sample weights are equal:

$$w_i = \frac{1}{14}$$

So ordinary Gini and weighted Gini give the same ranking initially.

Gini Impurity Formula

For a node with two classes Yes and No:

$$Gini = 1 - p_{Yes}^2 - p_{No}^2$$

Weighted Gini after a split:

$$Gini(Feature) = \sum_v \frac{n_v}{n} Gini(v)$$

where:

- v is a value of the feature,
- n_v is the number of samples in child node v ,
- n is the total number of samples.

Gini for Outlook

Outlook Value	Yes	No	Gini
Sunny	2	3	$1 - \left(\frac{2}{5}\right)^2 - \left(\frac{3}{5}\right)^2 = 0.48$
Overcast	4	0	$1 - \left(\frac{4}{4}\right)^2 - \left(\frac{0}{4}\right)^2 = 0$
Rain	3	2	$1 - \left(\frac{3}{5}\right)^2 - \left(\frac{2}{5}\right)^2 = 0.48$

Weighted Gini:

$$Gini(Outlook) = \frac{5}{14}(0.48) + \frac{4}{14}(0) + \frac{5}{14}(0.48) = 0.343$$

$Gini(Outlook) = 0.343$

Gini for Temperature

Counts:

Temperature = Hot : Yes = 2, No = 2

Temperature = Mild : Yes = 4, No = 2

Temperature = Cool : Yes = 3, No = 1

$$Gini(Hot) = 1 - \left(\frac{2}{4}\right)^2 - \left(\frac{2}{4}\right)^2 = 0.5$$

$$Gini(Mild) = 1 - \left(\frac{4}{6}\right)^2 - \left(\frac{2}{6}\right)^2 = 0.444$$

$$Gini(Cool) = 1 - \left(\frac{3}{4}\right)^2 - \left(\frac{1}{4}\right)^2 = 0.375$$

Gini for Humidity

Counts:

Humidity = High : Yes = 3, No = 4

Humidity = Normal : Yes = 6, No = 1

$$Gini(High) = 1 - \left(\frac{3}{7}\right)^2 - \left(\frac{4}{7}\right)^2 = 0.49$$

$$Gini(Normal) = 1 - \left(\frac{6}{7}\right)^2 - \left(\frac{1}{7}\right)^2 = 0.245$$

$$Gini(Humidity) = \frac{7}{14}(0.49) + \frac{7}{14}(0.245) = 0.3675$$

$Gini(Humidity) \approx 0.367$

Gini for Wind

Counts:

Wind = Strong : Yes = 3, No = 3

Wind = Weak : Yes = 6, No = 2

$$Gini(Strong) = 1 - \left(\frac{3}{6}\right)^2 - \left(\frac{3}{6}\right)^2 = 0.5$$

$$Gini(Weak) = 1 - \left(\frac{6}{8}\right)^2 - \left(\frac{2}{8}\right)^2 = 0.375$$

$$Gini(Wind) = \frac{6}{14}(0.5) + \frac{8}{14}(0.375) = 0.429$$

First Stump Selection

Feature	Weighted Gini
Outlook	0.343
Temperature	0.441
Humidity	0.367
Wind	0.429

Outlook has the lowest Gini impurity

Therefore, the first stump is:

Outlook

First Stump: Outlook

For the first stump:

Outlook = Sunny : Yes = 2, No = 3

Outlook = Overcast : Yes = 4, No = 0

Outlook = Rain : Yes = 3, No = 2

Therefore, stump predictions are:

Sunny $\Rightarrow$ No

Overcast $\Rightarrow$ Yes

Rain $\Rightarrow$ Yes

Total Error of First Stump

The stump predicts:

Sunny $\Rightarrow$ *No*, *Overcast* $\Rightarrow$ *Yes*, *Rain* $\Rightarrow$ *Yes*

Misclassified samples:

- Sunny branch: 2 Yes samples are misclassified.
- Overcast branch: 0 samples are misclassified.
- Rain branch: 2 No samples are misclassified.

Each sample has weight:

$$\frac{1}{14}$$

Total error:

$$err = 2 \left(\frac{1}{14} \right) + 0 + 2 \left(\frac{1}{14} \right) = \frac{4}{14} = \frac{2}{7} = 0.286$$

$err = 0.286$

Amount of Say for First Stump

Formula:

$$\alpha = \frac{1}{2} \ln \left(\frac{1 - \text{err}}{\text{err}} \right)$$

Here:

$$\text{err} = \frac{2}{7}$$

So,

$$\alpha = \frac{1}{2} \ln \left(\frac{1 - \frac{2}{7}}{\frac{2}{7}} \right)$$

$$\alpha = \frac{1}{2} \ln \left(\frac{\frac{5}{7}}{\frac{2}{7}} \right)$$

$$\alpha = \frac{1}{2} \ln \left(\frac{5}{2} \right)$$

Update the Sample Weights

For incorrectly classified samples:

$$w_i^{new} = w_i e^{\alpha}$$

For correctly classified samples:

$$w_i^{new} = w_i e^{-\alpha}$$

Here:

$$w_i = \frac{1}{14} = 0.0714$$

$$\alpha \approx 0.46$$

Incorrectly classified:

$$w_i^{new} = 0.0714 e^{0.46} \approx 0.113$$

Meaning of Weight Update

Sample Type	Weight Change	Meaning
Correctly classified	Decreases	Less attention next time
Incorrectly classified	Increases	More attention next time

Core AdaBoost Idea

AdaBoost focuses the next stump on the examples that the previous stump got wrong.

Normalize Updated Weights

Suppose the updated weights are:

0.045 for correct samples

0.113 for wrong samples

Then normalized weight is:

$$w_i^{normalized} = \frac{w_i^{new}}{\sum_j w_j^{new}}$$

Why?

The total weight should sum to 1 before the next round.

Weighted Resampling

After normalization, AdaBoost may create a new training dataset by sampling according to the updated weights.

- Samples with high weights are more likely to be selected.
- Misclassified samples may appear multiple times.
- Correctly classified samples may appear fewer times.

Important

This is different from Random Forest bootstrapping. In AdaBoost, sampling is influenced by sample weights.

Cumulative Weight and Random Sampling

To sample using weights:

- 1 Compute normalized weights.
- 2 Compute cumulative normalized weights.
- 3 Generate random numbers between 0 and 1.
- 4 Select the sample whose cumulative range contains the random number.

Example:

$$0.000 < r \leq 0.051 \Rightarrow \text{Day1}$$

$$0.051 < r \leq 0.102 \Rightarrow \text{Day2}$$

$$0.255 < r \leq 0.377 \Rightarrow \text{Day6}$$

Why Misclassified Samples Appear More Often

If a sample has large normalized weight, it occupies a larger interval in the cumulative weight table.

Large Weight $\Rightarrow$ Large Interval

Large Interval $\Rightarrow$ Higher Chance of Selection

Point

AdaBoost does not randomly focus on hard samples. It mathematically increases their probability of selection.

Second Stump

After resampling or reweighting, AdaBoost trains the next stump.

The next stump is trained on a distribution where hard samples have more importance.

From the slide example, after reweighting and resampling, the next best stump is based on:

Wind

The second stump has amount of say:

$$\alpha \approx 0.66$$

Final Prediction: Weighted Voting

Suppose we have two stumps:

$$\text{Stump}_1 : \text{Outlook}, \quad \alpha_1 = 0.45$$

$$\text{Stump}_2 : \text{Wind}, \quad \alpha_2 = 0.66$$

For a new sample:

Outlook = Sunny, Temperature = Mild, Humidity = Normal, Wind = Weak

Each stump gives a prediction.

- If a stump predicts Yes, add its α to Yes vote.
- If a stump predicts No, add its α to No vote.

Example of Weighted Voting

Assume:

$$\text{Stump}_1 \Rightarrow \text{No}, \quad \alpha_1 = 0.45$$

$$\text{Stump}_2 \Rightarrow \text{Yes}, \quad \alpha_2 = 0.66$$

Weighted vote:

$$\text{Vote}(\text{No}) = 0.45$$

$$\text{Vote}(\text{Yes}) = 0.66$$

Since:

$$0.66 > 0.45$$

Final prediction:

Yes

Important

This is not an unusual situation. The stump with lower amount of samples may influence

AdaBoost Algorithm Summary

- 1 Start with equal sample weights.
- 2 Train a weak learner.
- 3 Calculate its weighted error.
- 4 Calculate its amount of say.
- 5 Increase weights of misclassified samples.
- 6 Decrease weights of correctly classified samples.
- 7 Normalize weights.
- 8 Repeat for multiple weak learners.
- 9 Use weighted voting for final prediction.

Why AdaBoost Works

AdaBoost works because it repeatedly asks:

Which samples are still hard to classify?

Then it gives those samples more importance.

Intuition

Each new weak learner corrects the mistakes of the previous learners.

Model₁ → Mistakes

Model₂ → Focuses on those mistakes

Model₃ → Focuses on remaining mistakes

Strengths of AdaBoost

- Simple and powerful ensemble method.
- Often improves weak classifiers significantly.
- Automatically focuses on difficult samples.
- Less likely to overfit than a single weak learner.
- Provides interpretable weighted combination of weak models.

Limitations of AdaBoost

- Sensitive to noisy data and outliers.
- Mislabelled samples may receive very high weights.
- Sequential training can be slower than parallel methods.
- Performance depends on the quality of weak learners.

Important

If noisy samples are repeatedly misclassified, AdaBoost may focus too much on them.

Common Mistakes

- 1 Thinking AdaBoost uses ordinary majority voting.
- 2 Forgetting that misclassified samples get higher weights.
- 3 Confusing Random Forest bootstrapping with AdaBoost weighted sampling.
- 4 Choosing the stump with maximum Gini instead of minimum Gini.
- 5 Forgetting to normalize updated sample weights.
- 6 Thinking order does not matter in AdaBoost.

Important Formulas

$$Gini = 1 - \sum_{i=1}^k p_i^2$$

$$err_m = \frac{\sum_{i=1}^N w_i I(y_i \neq G_m(x_i))}{\sum_{i=1}^N w_i}$$

$$\alpha_m = \frac{1}{2} \ln \left(\frac{1 - err_m}{err_m} \right)$$

$$w_i^{new} = w_i e^{\alpha_m} \quad \text{if misclassified}$$

$$w_i^{new} = w_i e^{-\alpha_m} \quad \text{if correctly classified}$$

$$H(x) = \text{sign} \left(\sum_{m=1}^M \alpha_m G_m(x) \right)$$

Mini Practice Question

A weak learner misclassifies 3 samples out of 10.

All samples initially have equal weight.

- 1 What is the total error?
- 2 What is the amount of say?
- 3 Will the misclassified samples have larger or smaller weights after update?

$$err = \frac{3}{10} = 0.3$$

$$\alpha = \frac{1}{2} \ln \left(\frac{1 - 0.3}{0.3} \right)$$

$$\alpha = \frac{1}{2} \ln \left(\frac{0.7}{0.3} \right)$$

$$\alpha \approx 0.424$$

Summary

- AdaBoost is an adaptive boosting algorithm.
- It combines weak learners into a strong learner.
- Weak learners are often decision stumps.
- Misclassified samples receive more attention.
- Each weak learner receives an amount of say.
- Final prediction is made using weighted voting.

AdaBoost = Sequential Learning + Weight Update + Weighted Voting

Thank You